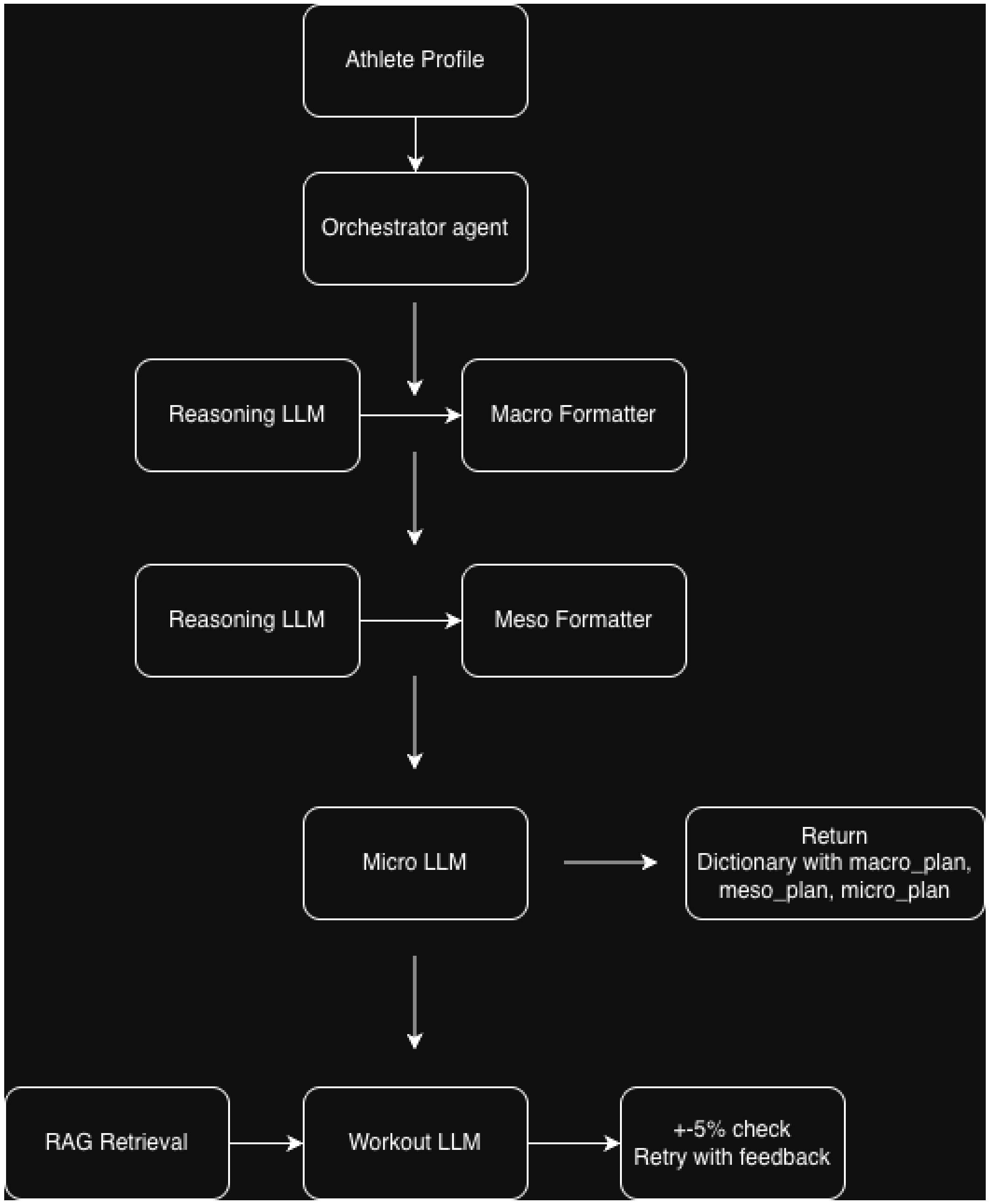
Swim Program Director

1. Problem and Motivation

The real life problem that this agentive system addresses is the lack of a concrete plan for swimmers who aren't in a college or club program. Even with University club swim, there aren't many resources available for swimmers who want to get better at their craft and reach their goals. This is what I faced during my time as president of the Boise State Swim Club from 2025-26. I had to do an ungodly amount of self-learning to understand how to actually train myself and be ready for my major meet in the Spring. This agentive system attempts to solve this problem by dividing up the planning of fitness cycles into 3 agents; a macrocycle planner, mesocycle planner, and microcycle planner. This project is a good fit for an agentive system because it allows each agent to be really good at their task. It also allows agents to break down the tasks and actually continue generating formatted cycles. The limitations that an LLM would have is just not being able to generate this much information in detail all at once. This is supposed to be a one stop shop for athletes to have this program build their fitness cycles while also generating personalized workouts.

2. System Architecture

The system architecture created for this program is displayed below:



It's fairly simplistic in nature because that's what the problem required. My original plan was to go with a more supervised pattern, however I soon realized that it was unnecessary for the task I was trying to automate given the amount of time and the priorities I had. The orchestrator sequentially delegates tasks to the agents below it. The reason this architecture was chosen was because each agent relies heavily on the agent above it. The mesocycle agent needs the macrocycle plan to do its job properly and the microcycle agent needs the mesocycle plans to do its job correctly. Unfortunately I realize this is a sequential process which I wasn't expecting in the project proposal and this means that the agents hand off work through the AgentState object, once the agent is done generating the plan, it stores the result inside AgentState and that is used in the next agents prompt.

3. Tool Integration

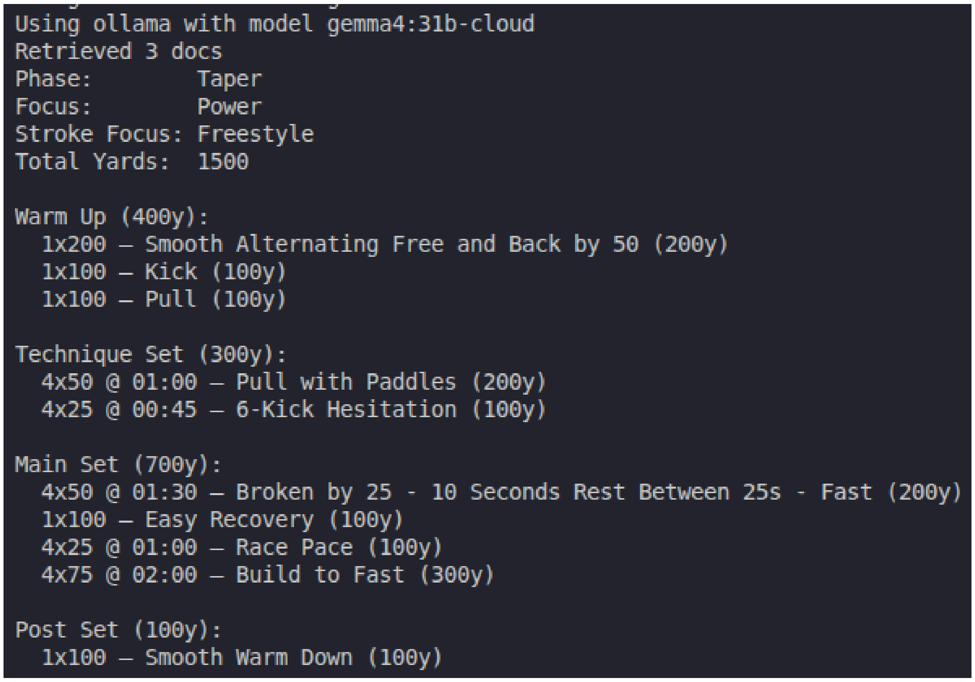
Unfortunately my solution currently doesn't use any tools. I think I set myself up incorrectly in the project proposal when I mentioned that there would be a tool to add up the yardage for each set. This is an ineffective way to compute that as since I had to make each set a JSON object, it's easier to compute the result and just query the agent again if the generated yardage isn't within 5% of the target yardage. This works really well and I did try some versions where I had the agent use a tool, however the agent *needs* to do this for every set so a tool here isn't necessary.

4. RAG Pipeline

My document corpus consists of thirty swim sets that I have written (more to be added in the future) all in the form of the JSON object that the program expects. I embed the structured JSON workouts with a chunk size of 2000 and overlap of 0 as the documents are short enough to fit within the chunk size. To get these JSON documents, I made a converter tool that allowed me to query an LLM to just convert the workouts I wrote into structured JSON, and that worked really well. To actually grab the documents, I have a workout_agent that takes a workout_stub and uses that stub to build a RAG query which then returns the top three results and puts it into the LLMs context. This allows the LLM to have structured examples to gain inspiration from. This agent loops a maximum of three times until a single workout is generated within 5% of the target yardage for that workout_stub.

5. Evaluation/Results

I am happy with some of the results and not happy with others. I think the macro agent does a great job of reasoning through the high level cycles and outputs some genuinely good information. However, the farther down the cycles you go, the more it starts to lose the story. The mesocycle generator does a decent job of generating microcycle stubs, however once we get to the microcycle stub everything falls apart. It may be that I am using a 12b parameter model for this or it could be my prompting strategy. The problem is whatever the first workout stub generated is, the next stubs are almost always exactly the same. This is frustrating because the whole point of the project is to essentially think through the problem, compress the solution into its most necessary information, then give the results to the next agent. However, I'll talk more about this in the reflection, but I think one of the major problems might be the compression of information. I am using a reasoning agent and structure agent for both the mesocycle and macrocycle generator and I think that may be where most of the useful information is being compressed and important stuff is being lost. The two examples of the system working vs. not working properly are in the workout_agent. If we use a large model (gemma4:31b), the model do a great job actually generating the target yardage because of the loop put in place to make sure there is a 5% error or less, however once we switch to a smaller model, the agent fails to generate within even 30% of an error to the target yardage. Both figures below are targeting a 1500 yard taper power freestyle set.



```
Using ollama with model gemma4:31b-cloud
Retrieved 3 docs
Phase:        Taper
Focus:        Power
Stroke Focus: Freestyle
Total Yards:  1500

Warm Up (400y):
  1x200 – Smooth Alternating Free and Back by 50 (200y)
  1x100 – Kick (100y)
  1x100 – Pull (100y)

Technique Set (300y):
  4x50 @ 01:00 – Pull with Paddles (200y)
  4x25 @ 00:45 – 6-Kick Hesitation (100y)

Main Set (700y):
  4x50 @ 01:30 – Broken by 25 - 10 Seconds Rest Between 25s - Fast (200y)
  1x100 – Easy Recovery (100y)
  4x25 @ 01:00 – Race Pace (100y)
  4x75 @ 02:00 – Build to Fast (300y)

Post Set (100y):
  1x100 – Smooth Warm Down (100y)
```

```
Using ollama with model gemma3:12b
Retrieved 3 docs
Phase:        Taper
Focus:        Power
Stroke Focus: Freestyle
Total Yards:  3300

Warm Up (800y):
  1x400 — Smooth Alternating Free and Back by 50 (400y)
  1x200 — Kick with Board (200y)
  1x200 — Pull with Buoy (200y)

Technique Set (700y):
  6x50 — Build with Fins - Focus on High Elbow (300y)
  6x50 — Catch-up Drill, focus on rotation (300y)
  4x25 — 6 Kicks, 1 Stroke, 6 Kicks, 1 Stroke (100y)

Main Set (1600y):
  4x100 — Broken 25s — Fast — 15 Seconds Rest (400y)
  1x200 — Easy Freestyle (200y)
  8x50 — Strong effort, 20 seconds rest (400y)
  1x300 — Maintain Tempo - focus on breathing (300y)
  4x75 — Max Effort, 30 seconds rest (300y)

Cool Down (200y):
  1x200 — Easy Swim (200y)
```

6. Reflection

From this project, the main thing I learned was to really think about how to force LLMs into responding with structured output. I don't think I fully succeeded because in the end I felt like I was more trying to do a lot of the LLMs job for it, however with all of the safety guardrails I wanted to implement, this seemed necessary. There were also a lot of times where I would get pretty frustrated that the LLMs weren't doing what I thought they should be doing and I learned to give it a couple minutes and rethink the problem again. Programming for agents is actually really hard. I have learned from this project because of the inconsistency. It's a bit like fine-tuning an LLM where you can't run the code 10 times in a single second. You really have to think about what things are possibly influencing the language model. Another thing I have learned is to set realistic expectations for a project like this. I thought I would have way more time than I did to actually work on this project and I am not super happy with where it is at the moment. Even though I did spend a lot of my time the last couple weeks working through this project, I had to do a lot of research to figure out how to do the task I was trying to automate manually, to understand what I was actually programming. This made development time explode as I was learning the theory and automating it at the same time. I hope to actually complete what I set out to do in the beginning over the summer because I still think there is hope for the project, particularly generating sets and macro cycles works very well. The middle section (meso and micro cycles) needs to be tweaked a lot. I also learned that in doing a solo project like this with that limited amount of time, I probably shouldn't have even tried doing a front end and added it later, however there isn't really anything I can do about that now. Overall, I think there is a lot of hope for this project and I'm happy with some of the results I got, however there is a lot more to be done.